1. This program will be a text based game where the player is venturing through a cave and runs into a monster. The player has the option to either fight the monster, or retreat out of the cave. Both the player and the monster will have an HP counter, additionally the player will have a money counter. If the player chooses to fight the monster a random number will generate, if that number is above a certain threshold the player will do some amount of damage to the monster. The monster is defeated when it runs out of HP. The monster will be able to attack the player back in the same way, however it will always have the chance to attack one last time after being defeated. If the player defeats the monster they will receive a certain amount of money, which they can choose to save or spend to increase their HP. The player will be able to fight as many monsters as they want until they are defeated, or choose to retreat.

2. The program will use a dictionary to store HP and money values, with player_hp, monster_hp, and player_money being the keys. There will be a custom function to handle the battle sequence, including the number generation and health updating. The whole game will run in a while loop that will stop when the player chooses to retreat, or is defeated. The user will input their actions—fighting or retreating, saving or spending—and see the results outputted on screen.

3.

Create a dictionary called game_values with item names as keys and prices as values
    "player_hp" maps to TBD (integer)
    "monster_hp" maps to TBD (integer)
    "player_money" maps to 0

Create a variable player_attk and set to 0
Create a variable monster_attk and set to 0

Define a function battle_sequence that takes player_hp and monster_hp
    Set player_attk to a random number
    If player_attk is greater than monster threshold subtract a TBD amount of HP from monster_hp
    Set monster_attk to a random number
    If monster_attk is greater than monster threshold subtract a TBD amount of HP from player_hp

Create a variable called run_game and set to True
Create a variable called player_retreat and set to False

Display “Welcome to Ultimate Fighter Battle 3001”
Ask the user to press enter to continue
Display “You enter a dark, damp cave with your trusty sword by your side”

Loop
    Display “While wandering the cave you stumble across a monster”
    Loop
        Ask the user to input 0 to fight or 1 retreat
        Try converting to an integer, if it works break out of the loop, if this doesn’t then tell the user they must input an integer 0 or 1
    If the user chooses to retreat then break out of the loop
    If the user chooses to fight call battle loop while both the player and monster have more than 0 HP

Call battle_sequence
If player_hp is zero, then display “Game Over” and break out of the loop
If monster_hp is zero, then increase player_money by TBD amount

Display player money
Ask if they would like to save, or increase HP for TBD amount
Display “Thank you for playing!”

4. The first test would be the user immediately retreating without fighting any monsters, this will make sure that this logic works as intended and does not make the user fight any monsters.
The second test would be to keep attacking monsters until the player runs out of HP, this would test to make sure that the logic to end the game when the player is defeated functions correctly. The would be to input a string when asked whether to fight or retreat, this will ensure the program can handle a bad input with breaking and needing to be reset.

5. I chose this topic because it seems like a fun way to learn how to write a program with lots of if-else logic, and has some added complexity with nested loops. I chose to create a function for the battle sequence as that will make the game loop easier to follow and debug, the if-else logic I tried to structure in a way that will make the game play easier to understand. The most challenging part was trying to handle all of the logic for the game without over complicating it and adding too many if-else statements. A dictionary works very well for this program as I can easily store and access the values I need to reference and update throughout the game in a human readable way.